

Trabalho Prático III

Profs. Cristiano Castro e João Pedro Campos

June 24, 2025

1 CONTROLE DE UM ROBÔ COLETOR DE LIXO COM APRENDIZADO POR REFORÇO (Q-LEARNING)



Figure 1.1: Robô coletor de lixo.

1.1 OBJETIVO

Aplicar os conceitos de Aprendizado por Reforço (Reinforcement Learning), em especial o algoritmo Q-Learning, para desenvolver um agente inteligente chamado Marvin, que deve aprender a tarefa de coletar lixo em um ambiente bidimensional simulado.

1.2 DESCRIÇÃO DO PROBLEMA

Você deverá desenvolver um agente virtual (robô) chamado Marvin, que vive em um ambiente simulado representado por uma grade bidimensional 50x50. O objetivo de Marvin é coletar o máximo possível de lixo, evitando obstáculos.

Cada célula da grade pode conter:

- +1: Lixo
- 0: Espaço vazio
- -1: Parede (obstáculo)

1.3 REGRAS DO AMBIENTE

- Marvin sempre inicia na posição (0, 0).
- Marvin possui sensores que percebem:
 - O conteúdo da célula atual
 - O conteúdo das quatro células adjacentes (norte, sul, leste, oeste).
- Marvin pode executar as seguintes ações:
 - Mover para norte.
 - Mover para sul.
 - Mover para leste.
 - Mover para oeste.
 - Mover para uma direção aleatória adjacente (N, S, L ou O).
 - Permanece parado.
 - Coletar lixo: tenta coletar lixo na célula atual.
- Cada episódio de treinamento tem duração de 1000 passos (iterações).

1.4 FUNÇÃO DE RECOMPENSA:

O desempenho de Marvin será avaliado de acordo com o seguinte esquema de recompensas:

- Coletar lixo com sucesso: +10 pontos.

- Tentar coletar lixo onde não há: -1 ponto.
- Colidir com uma parede (movimento inválido): -5 pontos.

1.5 TAREFAS

1. Modelar o problema como um MDP (Processo de Decisão de *Markov*):
 - Defina o espaço de estados.
 - Defina as ações possíveis.
 - Defina a função de recompensa.
2. Implementar o algoritmo de *Q-Learning* em Python.
3. Treinar o agente Marvin por múltiplos episódios (ex: 500 episódios).
4. Avaliar o desempenho do algoritmo usando uma estratégia para *exploration x exploitation* tal como ϵ -greedy.

1.6 ENTREGÁVEIS:

Vocês devem entregar:

- Código-fonte em Python (.ipynb), com comentários claros explicando a lógica.
- Relatório em PDF (máx. 5 páginas), contendo:
 - Descrição do ambiente e modelagem de estados.
 - Estratégias adotadas (representação da *Q-table*, escolha da política para seleção de ações - ϵ -greedy, etc.)
 - Gráficos de desempenho (pontuação por episódio).
 - A política ótima aprendida, derivada da *Q-table*:
Pode ser apresentada como um mapeamento estado $\rightarrow$ ação ótima; ou, como uma amostragem da política sobre partes do ambiente.
 - Discussão dos resultados.